Liquibase Snowflake Extension - Comprehensive User Guide

Overview

The Liquibase Snowflake Extension provides comprehensive support for Snowflake's unique database features, with **168+ test classes including 40 integration tests** providing robust support for Snowflake-specific objects, data types, and operations.

Key Features

- **Complete Object Support**: Warehouses, File Formats, Enhanced Schemas, Databases, Tables, Sequences
- **Advanced Data Types**: Semi-structured (VARIANT, OBJECT, ARRAY), Geospatial (GEOGRAPHY, GEOMETRY)
- **Full Lifecycle**: CREATE, ALTER, DROP operations with comprehensive property support
- **Schema Evolution**: Snapshot generation and diff detection for all Snowflake objects
- **Validation**: XSD schema validation with format-specific rules

Quick Reference: Supported Operations

Supported Operations Matrix

Object Type	CREATE	ALTER	DROP	Clone	Snapshot	Diff	Auto Rollback	Generate Changelog
Warehouse	✅	✅	✅	–	✅	✅	✅	✅
FileFormat	✅	✅	✅	–	✅	✅	✅	✅
Schema	✅	✅	✅	✅	✅	✅	✅	✅
Database	✅	✅	✅	✅	✅	✅	✅	✅
Sequence	✅	✅	✅	–	✅	✅	✅	✅
Table	✅	✅	✅	–	✅	✅	✅	✅

Notes:

- Objects marked with "–" for Clone are not supported by Snowflake for direct cloning
- Sequences are cloned automatically when their parent database/schema is cloned
- Auto Rollback: CREATE operations support automatic rollback (inverse DROP), DROP operations require manual rollback
- Generate Changelog: All objects support diff-changelog and generate-changelog via snapshot comparison

Operation Types by Object

Object Type	CREATE	ALTER	DROP	CLONE
Warehouse	Full property specification with size, clustering, auto-suspend, scaling policy	Size, cluster count, auto-suspend, scaling policy modifications	With optional `IF EXISTS`	Not supported (infrastructure objects)
FileFormat	Complete format configuration for CSV/JSON/Parquet/XML/Avro/ORC	SET/UNSET operations by property type and format	With optional `IF EXISTS`	Not supported (configuration objects)
Schema	Basic, managed access, data retention, collation settings	RENAME, SET, UNSET, ENABLE/DISABLE_MANAGED_ACCESS	With optional `IF EXISTS`, `CASCADE`, `RESTRICT`	Basic cloning support (`cloneFrom`, `fromSchema`)
Database	Standard, transient, Iceberg-enabled, data retention	Property modifications, retention settings	With optional `IF EXISTS`	Basic cloning support (`cloneFrom`, `fromDatabase`)
Sequence	Basic sequence with start, increment, min/max, cycling, ordering	Property modifications, increment changes	With optional `IF EXISTS`	Automatic with parent database/schema only
Table	Standard Liquibase tables with Snowflake namespace attributes	Property modifications via namespace attributes (clustering, change tracking)	Standard Liquibase operations	Supported by Snowflake but not yet implemented in extension

Changelog Format Support

The Liquibase Snowflake Extension supports all standard Liquibase changelog formats:

Supported Formats

- **XML** ✅ (Primary format with full XSD validation)
- **YAML** ✅ (All Snowflake operations supported)
- **JSON** ✅ (All Snowflake operations supported)
- **SQL** ✅ (Raw SQL with Snowflake syntax)

Quick Reference: Snowflake Objects in XML

```xml
<databaseChangeLog
    xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
    xmlns:snowflake="http://www.liquibase.org/xml/ns/snowflake">

    <!-- Warehouse -->
    <changeSet id="1" author="admin">
        <snowflake:createWarehouse warehouseName="ANALYTICS_WH"
                                   warehouseSize="MEDIUM"
                                   autoSuspend="300"/>
    </changeSet>

    <!-- Database -->
    <changeSet id="2" author="admin">
        <snowflake:createDatabase databaseName="ANALYTICS_DB"
                                  dataRetentionTimeInDays="7"/>
    </changeSet>
```

```xml
    <!-- Schema -->
    <changeSet id="3" author="admin">
        <snowflake:createSchema schemaName="RAW_DATA"
                                managedAccess="true"/>
    </changeSet>

    <!-- FileFormat -->
    <changeSet id="4" author="admin">
        <snowflake:createFileFormat fileFormatName="CSV_FORMAT"
                                    fileFormatType="CSV"
                                    fieldDelimiter=","
                                    skipHeader="1"/>
    </changeSet>

    <!-- Sequence -->
    <changeSet id="5" author="admin">
        <snowflake:createSequence sequenceName="ORDER_ID_SEQ"
                                  startValue="1000"
                                  incrementBy="1"/>
    </changeSet>

    <!-- Table with Snowflake attributes -->
    <changeSet id="6" author="admin">
        <createTable tableName="EVENTS"
                     snowflake:clusterBy="(EVENT_DATE)"
                     snowflake:changeTracking="true">
            <column name="ID" type="NUMBER(38,0)" autoIncrement="true"/>
            <column name="EVENT_DATA" type="VARIANT"/>
            <column name="EVENT_DATE" type="DATE"/>
        </createTable>
    </changeSet>
</databaseChangeLog>
```

Part 1: Database Objects Reference

1.1 Warehouse

Purpose: Virtual compute clusters for query processing with scaling and resource management.

Properties

Property	Type	Required	Default	Description
`warehouseName`	String	✅	-	Warehouse identifier
`warehouseSize`	Enum	–	`MEDIUM`	`XSMALL`, `SMALL`, `MEDIUM`, `LARGE`, `XLARGE`, `XXLARGE`, `XXXLARGE`, `X4LARGE`, `X5LARGE`, `X6LARGE`
`warehouseType`	Enum	–	`STANDARD`	`STANDARD`, `SNOWPARK-OPTIMIZED`
`autoSuspend`	Integer	–	`600`	Seconds before auto-suspend (0 = never, ≥60)
`autoResume`	Boolean	–	`true`	Enable automatic resume
`initiallySuspended`	Boolean	–	`false`	Whether warehouse should be initially suspended
`maxClusterCount`	Integer	–	`1`	Maximum clusters (1-10)
`minClusterCount`	Integer	–	`1`	Minimum clusters (1-10)
`scalingPolicy`	Enum	–	`STANDARD`	`STANDARD`, `ECONOMY`
`comment`	String	–	-	Warehouse description
`resourceMonitor`	String	–	-	Resource monitor name
`enableQueryAcceleration`	Boolean	–	`false`	Whether to enable query acceleration
`queryAccelerationMaxScaleFactor`	Integer	–	-	Query acceleration max scale factor (0-100)
`maxConcurrencyLevel`	Integer	–	-	Maximum concurrency level
`statementQueuedTimeoutInSeconds`	Integer	–	-	Statement queued timeout in seconds
`statementTimeoutInSeconds`	Integer	–	-	Statement timeout in seconds
`resourceConstraint`	String	–	-	Resource constraint (MEMORY_1X, MEMORY_2X, etc.)
`orReplace`	Boolean	–	`false`	Replace if exists
`ifNotExists`	Boolean	–	`false`	Create only if not exists

Note: Warehouses do not support cloning in Snowflake (infrastructure objects).

Usage Examples

```
<!-- Basic warehouse creation -->
<snowflake:createWarehouse warehouseName="ANALYTICS_WH"
                           warehouseSize="LARGE"
                           autoSuspend="300"/>

<!-- Advanced multi-cluster warehouse -->
<snowflake:createWarehouse warehouseName="ETL_WH"
                           warehouseSize="XLARGE"
                           minClusterCount="2"
                           maxClusterCount="8"
                           scalingPolicy="ECONOMY"
                           comment="ETL processing warehouse"/>

<!-- Alter warehouse properties -->
<snowflake:alterWarehouse warehouseName="ANALYTICS_WH"
                          autoSuspend="600"
```

```
                          warehouseSize="XLARGE"/>

<!-- Drop warehouse -->
<snowflake:dropWarehouse warehouseName="OLD_WH" ifExists="true"/>
```

Validation Rules

- `maxClusterCount >= minClusterCount`
- `autoSuspend` must be 0 (never suspend) or ≥60 seconds
- `orReplace` and `ifNotExists` are mutually exclusive

1.2 File Format

Purpose: Define data loading formats for CSV, JSON, Parquet, XML, Avro, ORC, and custom formats.

Properties

Property	Type	Required	Default	Description
`fileFormatName`	String	✅	-	File format identifier
`fileFormatType`	Enum	✅	-	`CSV`, `JSON`, `PARQUET`, `XML`, `AVRO`, `ORC`, `CUSTOM`

CSV-Specific Properties

Property	Type	Default	Description
`fieldDelimiter`	String	`,`	Field separator character
`recordDelimiter`	String	`\n`	Record separator
`skipHeader`	Integer	`0`	Number of header rows to skip
`fieldOptionallyEnclosedBy`	String	`"`	Field enclosure character
`escape`	String	`\\`	Escape character
`escapeUnenclosedField`	String	`\\`	Escape for unenclosed fields
`trimSpace`	Boolean	`false`	Trim whitespace
`errorOnColumnCountMismatch`	Boolean	`true`	Error on column mismatch

JSON-Specific Properties

Property	Type	Default	Description
`enableOctal`	Boolean	`false`	Enable octal number parsing
`allowDuplicate`	Boolean	`false`	Allow duplicate keys
`stripOuterArray`	Boolean	`false`	Strip outer array wrapper
`stripNullValues`	Boolean	`false`	Remove null values
`ignoreUTF8Errors`	Boolean	`false`	Ignore UTF-8 encoding errors

Parquet-Specific Properties

Property	Type	Default	Description
`snappyCompression`	Boolean	`true`	Use Snappy compression
`binaryAsText`	Boolean	`true`	Treat binary as text

XML-Specific Properties

Property	Type	Default	Description
`preserveSpace`	Boolean	`false`	Preserve whitespace
`stripOuterElement`	Boolean	`false`	Strip root element
`disableSnowflakeData`	Boolean	`false`	Disable Snowflake metadata
`disableAutoConvert`	Boolean	`false`	Disable auto type conversion

Universal Properties

Property	Type	Default	Description
`compression`	Enum	`AUTO`	`AUTO`, `GZIP`, `BZ2`, `BROTLI`, `ZSTD`, `DEFLATE`, `RAW_DEFLATE`, `NONE`
`dateFormat`	String	`AUTO`	Date parsing format
`timeFormat`	String	`AUTO`	Time parsing format
`timestampFormat`	String	`AUTO`	Timestamp parsing format
`binaryFormat`	Enum	`HEX`	`HEX`, `BASE64`, `UTF8`
`nullIf`	String	`\\N`	String representing NULL

Additional Format Properties

Property	Type	Default	Description
`replaceInvalidCharacters`	Boolean	`false`	Replace invalid UTF-8 characters
`skipByteOrderMark`	Boolean	`false`	Skip byte order mark
`encoding`	String	`UTF8`	Character encoding
`fileExtension`	String	-	Expected file extension
`multiLine`	Boolean	`false`	Support multi-line records
`parseHeader`	Boolean	`false`	Parse header row

Note: FileFormats do not support cloning in Snowflake (configuration objects).

Usage Examples

```
<!-- CSV file format -->
<snowflake:createFileFormat fileFormatName="CSV_FORMAT"
                            fileFormatType="CSV"
                            fieldDelimiter=","
                            skipHeader="1"
                            fieldOptionallyEnclosedBy="&quot;"
                            compression="GZIP"/>

<!-- JSON file format with advanced options -->
<snowflake:createFileFormat fileFormatName="JSON_FORMAT"
                            fileFormatType="JSON"
                            stripOuterArray="true"
                            dateFormat="YYYY-MM-DD"
                            timestampFormat="YYYY-MM-DD HH24:MI:SS"/>

<!-- Parquet format -->
<snowflake:createFileFormat fileFormatName="PARQUET_FORMAT"
                            fileFormatType="PARQUET"
                            snappyCompression="true"
                            binaryAsText="false"/>

<!-- Alter file format -->
<snowflake:alterFileFormat fileFormatName="CSV_FORMAT"
                           operationType="SET"
                           compression="BROTLI"
                           trimSpace="true"/>

<!-- Drop file format -->
<snowflake:dropFileFormat fileFormatName="OLD_FORMAT" ifExists="true"/>
```

Format-Specific Validation

- CSV: `fieldDelimiter` cannot equal `escape` character
- JSON: `stripOuterArray` requires array input data
- Parquet: Limited compression options (SNAPPY, GZIP, LZO)
- Universal: `compression` availability depends on format type

1.3 Enhanced Schema

Purpose: Snowflake schemas with managed access, data retention, and cloning capabilities.

Properties

Property	Type	Required	Default	Description
`schemaName`	String	✅	-	Schema identifier
`managedAccess`	Boolean	–	`false`	Enable managed access
`dataRetentionTimeInDays`	String	–	`"1"`	Time Travel retention (0-90)
`comment`	String	–	-	Schema description
`cloneFrom`	String	–	-	Source schema for cloning
`fromSchema`	String	–	-	Source schema for cloning (alias for cloneFrom)
`defaultDdlCollation`	String	–	-	Default DDL collation
`externalVolume`	String	–	-	External volume name
`classificationProfile`	String	–	-	Classification profile name
`tag`	String	–	-	Tag assignment
`replaceInvalidCharacters`	String	–	-	Replace invalid characters setting
`storageSerializationPolicy`	String	–	-	Storage serialization policy
`orReplace`	Boolean	–	`false`	Replace if exists
`ifNotExists`	Boolean	–	`false`	Create only if not exists

ALTER Operations

Operation	Description
`RENAME`	Change schema name
`SET`	Modify properties
`UNSET`	Remove property values
`ENABLE_MANAGED_ACCESS`	Enable managed access
`DISABLE_MANAGED_ACCESS`	Disable managed access

Usage Examples

```
<!-- Basic schema creation -->
<snowflake:createSchema schemaName="ANALYTICS"
                        comment="Analytics data schema"/>

<!-- Schema with managed access -->
<snowflake:createSchema schemaName="SENSITIVE_DATA"
                        managedAccess="true"
                        dataRetentionTimeInDays="7"
                        defaultDdlCollation="utf8"/>

<!-- Clone schema -->
<snowflake:createSchema schemaName="ANALYTICS_BACKUP"
                        catalogName="PROD_DB"
                        cloneFrom="ANALYTICS"
                        comment="Backup of analytics schema"/>

<!-- Alter schema operations -->
<snowflake:alterSchema schemaName="ANALYTICS"
                       operationType="SET"
                       dataRetentionTimeInDays="30"
                       comment="Updated analytics schema"/>

<snowflake:alterSchema schemaName="OLD_SCHEMA"
                       operationType="RENAME"
                       newName="NEW_SCHEMA"/>

<!-- Drop schema -->
<snowflake:dropSchema schemaName="TEMP_SCHEMA" ifExists="true"/>
```

1.4 Database

Purpose: Top-level database containers with Time Travel, transient options, and Iceberg support.

Properties

Properties

Property	Type	Required	Default	Description
`databaseName`	String	✅	-	Database identifier
`transient`	Boolean	–	`false`	Transient database (no Time Travel)
`dataRetentionTimeInDays`	String	–	`"1"`	Time Travel retention (0-90)
`comment`	String	–	-	Database description
`cloneFrom`	String	–	-	Source database to clone from
`fromDatabase`	String	–	-	Alternative source database name for cloning
`tag`	String	–	-	Tag to apply to the database
`externalVolume`	String	–	-	External volume for Iceberg tables
`catalog`	String	–	-	Catalog integration for Iceberg tables
`replaceInvalidCharacters`	Boolean	–	-	Replace invalid UTF-8 characters
`storageSerializationPolicy`	String	–	-	Storage serialization policy
`catalogSync`	String	–	-	Snowflake Open Catalog integration name
`catalogSyncNamespaceMode`	String	–	-	Catalog sync namespace mode
`catalogSyncNamespaceFlattenDelimiter`	String	–	-	Namespace flatten delimiter
`defaultDdlCollation`	String	–	-	Default DDL collation
`orReplace`	Boolean	–	`false`	Replace if exists
`ifNotExists`	Boolean	–	`false`	Create only if not exists

Usage Examples

```
<!-- Standard database -->
<snowflake:createDatabase databaseName="PRODUCTION"
                          dataRetentionTimeInDays="7"
                          comment="Production database"/>

<!-- Transient database (no Time Travel) -->
<snowflake:createDatabase databaseName="STAGING"
                          transient="true"
                          comment="Staging environment"/>

<!-- Iceberg-enabled database -->
<snowflake:createDatabase databaseName="ICEBERG_DB"
                          catalog="ICEBERG_CATALOG"
                          externalVolume="ICEBERG_VOLUME"/>

<!-- Clone database -->
<snowflake:createDatabase databaseName="DEV_CLONE"
                          cloneFrom="PRODUCTION"
                          comment="Clone of production database"/>

<!-- Alter database -->
```

```
<!-- Alter database -->
<snowflake:alterDatabase databaseName="PRODUCTION"
                         dataRetentionTimeInDays="30"/>
```

1.5 Sequence

Purpose: Snowflake sequences with enhanced ordering and caching features.

Properties

Property	Type	Required	Default	Description
`sequenceName`	String	✅	-	Sequence identifier
`startValue`	BigInteger	–	`1`	Starting value
`incrementBy`	BigInteger	–	`1`	Increment step
`minValue`	BigInteger	–	-	Minimum value
`maxValue`	BigInteger	–	-	Maximum value
`cycle`	Boolean	–	`false`	Cycle when limit reached
`ordered`	Boolean	–	`false`	Guarantee ordered values
`comment`	String	–	-	Sequence description
`orReplace`	Boolean	–	`false`	Replace if exists
`ifNotExists`	Boolean	–	`false`	Create only if not exists

Note: Sequences do not support direct cloning but are cloned automatically when their parent database/schema is cloned.

Usage Examples

```
<!-- Basic sequence -->
<snowflake:createSequence sequenceName="USER_ID_SEQ"
                          startValue="1000"
                          incrementBy="1"/>

<!-- Ordered sequence with cycling -->
<snowflake:createSequence sequenceName="BATCH_ID_SEQ"
                          startValue="1"
                          incrementBy="1"
                          maxValue="999999"
                          cycle="true"
                          ordered="true"/>


<!-- Alter sequence -->
```

```
<!-- Alter Sequence -->
<snowflake:alterSequence sequenceName="USER_ID_SEQ"
                         incrementBy="10"
                         comment="Updated increment"/>
```

1.6 Table (Enhanced with Snowflake Attributes)

Purpose: Standard Liquibase tables enhanced with Snowflake-specific features via namespace attributes.

Snowflake-Specific Table Attributes

Attribute	Type	Description
`snowflake:clusterBy`	String	Clustering key specification
`snowflake:changeTracking`	Boolean	Enable change tracking
`snowflake:dataRetentionTimeInDays`	String	Time Travel retention
`snowflake:enableSchemaEvolution`	Boolean	Schema evolution support
`snowflake:tableType`	Enum	`PERMANENT`, `TEMPORARY`, `TRANSIENT`

Usage Examples

```
<!-- Table with Snowflake clustering -->
<createTable tableName="SALES_DATA"
             snowflake:clusterBy="(DATE, REGION)"
             snowflake:changeTracking="true"
             snowflake:dataRetentionTimeInDays="30">
    <column name="ID" type="NUMBER(38,0)" autoIncrement="true"/>
    <column name="DATE" type="DATE"/>
    <column name="REGION" type="VARCHAR(50)"/>
    <column name="AMOUNT" type="NUMBER(12,2)"/>
</createTable>

<!-- Transient table -->
<createTable tableName="TEMP_STAGING"
             snowflake:tableType="TRANSIENT">
    <column name="DATA" type="VARIANT"/>
    <column name="LOADED_AT" type="TIMESTAMP_NTZ"/>
</createTable>

<!-- Alter table Snowflake properties -->
<snowflake:alterTable tableName="SALES_DATA"
                      clusterBy="(DATE, REGION, CUSTOMER_ID)"
                      setChangeTracking="false"/>
```

Part 2: Snowflake Data Types

2.1 Semi-Structured Data Types

Data Type	Purpose	Usage Example	Use Cases
VARIANT	JSON-like semi-structured data storage	`<column name="JSON_DATA" type="VARIANT"/>`	API responses, flexible schemas, JSON documents, mixed data types
OBJECT	Key-value pair storage	`<column name="METADATA" type="OBJECT"/>`	Configuration settings, user preferences, metadata storage
ARRAY	Ordered list storage	`<column name="TAGS" type="ARRAY"/>`	Tag lists, ordered collections, hierarchical data

2.2 Geospatial Data Types

Data Type	Purpose	Usage Example	Use Cases
GEOGRAPHY	Earth-based geospatial data (WGS84)	`<column name="LOCATION" type="GEOGRAPHY"/>`	GPS coordinates, global mapping, location tracking, GIS applications
GEOMETRY	Planar geospatial data	`<column name="SHAPE" type="GEOMETRY"/>`	CAD drawings, floor plans, 2D graphics, geometric calculations

2.3 Enhanced Standard Types

Data Type	Purpose	Usage Example	Use Cases
TEXT	Large text storage (16MB limit)	`<column name="DESCRIPTION" type="TEXT"/>`	Long descriptions, document content, large text fields, rich text data
BINARY	Binary data storage (8MB limit)	`<column name="FILE_DATA" type="BINARY"/>`	File uploads, images, encrypted data, binary documents
TIMESTAMP_NTZ	Timestamp without timezone	`<column name="EVENT_TIME" type="TIMESTAMP_NTZ"/>`	Log timestamps, event tracking, system timestamps, UTC data

---

Part 3: Additional Liquibase Functionality

Rollback Support

- **Automatic Rollback**: CREATE operations automatically generate inverse DROP statements
- **Manual Rollback**: DROP and complex ALTER operations require custom `<rollback>` tags
- **Time-based Rollback**: Rollback to specific timestamps using Liquibase tags

- **Time-based Rollback**: Rollback to specific timestamps using Liquibase tags

Schema Evolution

- **Diff-Changelog**: Compare databases and generate changesets for differences
- **Generate-Changelog**: Create changelogs from existing database schemas
- **Snapshot Comparison**: All Snowflake objects support snapshot-based diff generation

Database Validation

- **Preconditions**: Standard Liquibase preconditions work with all Snowflake objects
- **Data Validation**: Custom preconditions can validate Snowflake-specific object properties
- **Environment Checks**: Warehouse size, file format validation, schema permissions

Change Tracking

- **Tagging**: Standard Liquibase tagging for marking database states
- **Change Log History**: Full tracking of all Snowflake object changes
- **Rollback Points**: Tag-based rollback to specific schema versions

---

Part 4: Advanced Features

4.1 Snapshot and Diff Support

The extension provides complete snapshot generation and diff detection with dual architecture patterns:

Extension Object Architecture Patterns

Schema-Level Objects (FileFormat, Stage, Pipe)

- **Parent**: `Schema.class`
- **Discovery**: INFORMATION_SCHEMA queries with parameterized schemas
- **Pattern**: Standard Liquibase snapshot generators work reliably
- **Examples**: FileFormat, future Stage/Pipe implementations

Account-Level Objects (Warehouse, User, Role)

- **Parent**: `Account.class`
- **Discovery**: SHOW commands (no schema parameters)
- **Pattern**: Requires unified extensibility framework (SnowflakeExtensionDiffGeneratorSimple)
- **Examples**: Warehouse (implemented), future User/Role objects

Snapshot Features

Snapshot Features

- **Object Discovery**: Automatic detection with architecture-appropriate patterns
- **Property Extraction**: Complete property capture from INFORMATION_SCHEMA
- **Relationship Mapping**: Schema hierarchies and account-level dependencies
- **State Comparison**: Point-in-time snapshots for diff generation

Diff Generation

- **Schema Changes**: Automatic changeset generation from snapshots
- **Property Changes**: Granular ALTER operations for property modifications
- **Object Lifecycle**: CREATE for missing objects, DROP for unexpected objects
- **Validation**: XSD schema validation of generated changesets

4.2 Time Travel and Cloning

Time Travel Support

```
<!-- Clone schema -->
<snowflake:createSchema schemaName="HISTORICAL_DATA"
                        fromSchema="CURRENT_DATA"
                        comment="Historical data clone"/>

<!-- Clone database -->
<snowflake:createDatabase databaseName="PRE_MIGRATION"
                          cloneFrom="PRODUCTION"
                          comment="Pre-migration database clone"/>
```

Data Retention Configuration

```
<!-- Set retention for database -->
<snowflake:alterDatabase databaseName="ANALYTICS"
                         dataRetentionTimeInDays="90"/>

<!-- Set retention for schema -->
<snowflake:alterSchema schemaName="CRITICAL_DATA"
                       operationType="SET"
                       dataRetentionTimeInDays="30"/>
```

4.3 Validation and Error Handling

Mutual Exclusivity Rules

- `orReplace` and `ifNotExists` cannot be used together
- SET and UNSET operations are mutually exclusive in ALTER commands

- Compression types are format-specific in file formats

Range Validations

- Warehouse auto-suspend: 60-86400 seconds
- Data retention: 0-90 days
- Cluster counts: 1-10 clusters
- Skip header: 0+ rows

Format-Specific Validations

- CSV: Field delimiter ≠ escape character
- JSON: Date format compatibility with data
- Parquet: Compression support validation

Part 5: Best Practices and Integration

5.1 Changelog Organization

```xml
<?xml version="1.0" encoding="UTF-8"?>
<databaseChangeLog
    xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
    xmlns:snowflake="http://www.liquibase.org/xml/ns/snowflake">

    <!-- 1. Infrastructure Setup -->
    <changeSet id="001-create-warehouse" author="admin">
        <snowflake:createWarehouse warehouseName="ANALYTICS_WH"
                                   warehouseSize="MEDIUM"
                                   autoSuspend="600"/>
    </changeSet>

    <!-- 2. Database Structure -->
    <changeSet id="002-create-database" author="admin">
        <snowflake:createDatabase databaseName="ANALYTICS_DB"
                                  dataRetentionTimeInDays="7"/>
    </changeSet>

    <!-- 3. Schema Setup -->
    <changeSet id="003-create-schema" author="admin">
        <snowflake:createSchema schemaName="RAW_DATA"
                                managedAccess="true"/>
    </changeSet>

    <!-- 4. File Formats -->
```

```
    <changeSet id="004-create-formats" author="admin">
        <snowflake:createFileFormat fileFormatName="JSON_FORMAT"
                                    fileFormatType="JSON"
                                    stripOuterArray="true"/>
    </changeSet>

    <!-- 5. Tables and Data -->
    <changeSet id="005-create-tables" author="developer">
        <createTable tableName="EVENTS"
                     snowflake:clusterBy="(EVENT_DATE)"
                     snowflake:changeTracking="true">
            <column name="ID" type="NUMBER(38,0)" autoIncrement="true"/>
            <column name="EVENT_DATA" type="VARIANT"/>
            <column name="EVENT_DATE" type="DATE"/>
        </createTable>
    </changeSet>
</databaseChangeLog>
```

5.2 Environment-Specific Configuration

Use Liquibase contexts to target different environments with appropriate resource sizing:

Context-Based Environment Configuration

```
<!-- Development Environment - Small resources, fast teardown -->
<changeSet id="001-dev-warehouse" author="admin" context="dev">
    <snowflake:createWarehouse warehouseName="DEV_WH"
                               warehouseSize="XSMALL"
                               autoSuspend="60"
                               comment="Development warehouse - auto-suspend quickly"/>
</changeSet>

<changeSet id="002-dev-database" author="admin" context="dev">
    <snowflake:createDatabase databaseName="DEV_DB"
                              transient="true"
                              comment="Development database - no Time Travel"/>
</changeSet>

<!-- Staging Environment - Medium resources, moderate retention -->
<changeSet id="001-staging-warehouse" author="admin" context="staging">
    <snowflake:createWarehouse warehouseName="STAGING_WH"
                               warehouseSize="SMALL"
                               autoSuspend="300"
                               maxClusterCount="2"
                               comment="Staging warehouse - moderate scaling"/>
</changeSet>

<changeSet id="002-staging-database" author="admin" context="staging">
```

```
    <snowflake:createDatabase databaseName="STAGING_DB"
                              dataRetentionTimeInDays="7"
                              comment="Staging database - 1 week retention"/>
</changeSet>

<!-- Production Environment - High availability, full retention -->
<changeSet id="001-prod-warehouse" author="admin" context="prod">
    <snowflake:createWarehouse warehouseName="PROD_WH"
                               warehouseSize="LARGE"
                               minClusterCount="2"
                               maxClusterCount="8"
                               scalingPolicy="STANDARD"
                               autoSuspend="600"
                               comment="Production warehouse - high availability"/>
</changeSet>

<changeSet id="002-prod-database" author="admin" context="prod">
    <snowflake:createDatabase databaseName="PROD_DB"
                              dataRetentionTimeInDays="90"
                              comment="Production database - full Time Travel retention"/>
</changeSet>
```

Deployment Commands by Environment

```
# Deploy to development (includes common changes + dev-specific)
liquibase update --context-filter=dev

# Deploy to staging (includes common changes + staging-specific)
liquibase update --context-filter=staging

# Deploy to production (includes common changes + prod-specific)
liquibase update --context-filter=prod

# Deploy only common changes (no context specified)
liquibase update

# Deploy to non-production environments only
liquibase update --context-filter="!prod"
```

Mixed Context Strategy

```
<!-- Common infrastructure - no context (deployed everywhere) -->
<changeSet id="100-common-file-format" author="admin">
    <snowflake:createFileFormat fileFormatName="JSON_FORMAT"
                                fileFormatType="JSON"
                                stripOuterArray="true"/>

</changeSet>
```

```
<!-- Environment-specific schemas -->
<changeSet id="101-dev-schema" author="admin" context="dev">
    <snowflake:createSchema schemaName="DEV_ANALYTICS" managedAccess="false"/>
</changeSet>

<changeSet id="101-prod-schema" author="admin" context="prod">
    <snowflake:createSchema schemaName="PROD_ANALYTICS"
                            managedAccess="true"
                            dataRetentionTimeInDays="30"/>
</changeSet>
```

5.3 Migration Strategies

Blue-Green Deployments

```
<!-- Create new schema version -->
<snowflake:createSchema schemaName="APP_V2"
                        fromSchema="APP_V1"/>

<!-- Update with new structure -->
<changeSet id="migrate-v2" author="developer">
    <!-- Apply changes to APP_V2 -->
</changeSet>

<!-- Switch traffic (external to Liquibase) -->
<!-- Drop old version -->
<snowflake:dropSchema schemaName="APP_V1"/>
```

Rollback Strategies

```
<!-- Point-in-time recovery -->
<snowflake:createSchema schemaName="ROLLBACK_SCHEMA"
                        fromSchema="PRODUCTION_SCHEMA"
                        comment="Rollback schema clone"/>
```

5.4 Development Timeline Planning

For development teams planning new Snowflake object support:

Simple Objects (1-2 Days Development)

- **Examples**: Database, Sequence operations
- **Scope**: 5-10 properties with standard patterns

- **Timeline**: Day 1 (Implementation), Day 2 (Testing & XSD)

Moderate Objects (3-5 Days Development)

- **Examples**: Schema, Warehouse, User, Role
- **Scope**: 10-15 properties, may need account-level patterns
- **Timeline**: 3 days implementation, 2 days testing/integration

Complex Objects (1-2 Weeks Development)

- **Examples**: FileFormat, Stage, Pipe, View
- **Scope**: 25+ properties with format-specific logic
- **Timeline**: Week 1 (Core implementation), Week 2 (Advanced features & testing)

Part 6: Testing and Validation

6.1 Test Coverage Status

- **Overall Extension Coverage**: Comprehensive (168+ test classes)
- **Integration Tests**: 40 classes with live Snowflake connections
- **Change Operations**: Fully tested CREATE/ALTER/DROP operations
- **Database Objects**: Complete object model validation
- **Snapshot Generators**: 8 generators with complete SQL assertion testing
- **Parallel Testing**: Schema isolation enables 4x faster CI/CD execution

6.2 Validation Commands

```
# Validate extension compilation
mvn compile -q

# Run all tests
mvn test -q

# Test specific object types
mvn test -Dtest="*FileFormat*Test*" -q
mvn test -Dtest="*Warehouse*Test*" -q
mvn test -Dtest="*Schema*Test*" -q

# Generate coverage report (requires full test execution)
mvn test jacoco:report
open target/site/jacoco/index.html


# Test architecture patterns (parallel execution)
```

```
mvn test -Dtest="*IntegrationTest" -q  # 40 integration tests in parallel
mvn test -Dtest="*SnapshotGenerator*Test*" -q  # 8 snapshot generators
```

6.3 Advanced Liquibase Operations

Diff and Generate Changelog

```
# Generate changelog from existing Snowflake database
liquibase generate-changelog --changelog-file=snowflake-baseline.xml

# Compare two databases and generate diff changelog
liquibase diff-changelog --reference-url=jdbc:snowflake://prod.snowflakecomputing.com \
  --url=jdbc:snowflake://dev.snowflakecomputing.com \
  --changelog-file=prod-to-dev-diff.xml

# Include only specific object types
liquibase diff-changelog --diff-types=warehouse,fileFormat,schema \
  --changelog-file=infrastructure-diff.xml
```

Rollback Operations

```
# Rollback to specific tag
liquibase rollback --tag=v1.0.0

# Rollback specific number of changesets
liquibase rollback-count --count=3

# Generate rollback SQL for review
liquibase rollback-sql --tag=v1.0.0 > rollback-review.sql
```

Custom Rollback Example

```
<changeSet id="create-warehouse-with-rollback" author="admin">
    <snowflake:createWarehouse warehouseName="ANALYTICS_WH" warehouseSize="LARGE"/>
    <rollback>
        <snowflake:dropWarehouse warehouseName="ANALYTICS_WH" ifExists="true"/>
    </rollback>
</changeSet>
```

6.4 Integration Testing

The extension includes comprehensive integration tests with live Snowflake connections:

```
# Test configuration: src/test/resources/liquibase.sdk.local.yaml
```

```yaml
liquibase:
  sdk:
    testSystem:
      snowflake:
        url: "jdbc:snowflake://account.snowflakecomputing.com/?db=TEST_DB&warehouse=TEST_WH"
        username: "test_user"
        password: "test_password"
```

Appendix: XSD Schema Reference

The extension provides complete XSD schema validation with 800+ lines of schema definitions:

```xml
<!-- Warehouse element definition -->
<xsd:element name="createWarehouse">
    <xsd:complexType>
        <xsd:attribute name="warehouseName" type="xsd:string" use="required"/>
        <xsd:attribute name="warehouseSize" use="optional">
            <xsd:simpleType>
                <xsd:restriction base="xsd:string">
                    <xsd:enumeration value="XSMALL"/>
                    <xsd:enumeration value="SMALL"/>
                    <xsd:enumeration value="MEDIUM"/>
                    <xsd:enumeration value="LARGE"/>
                    <!-- ... more size options -->
                </xsd:restriction>
            </xsd:simpleType>
        </xsd:attribute>
        <!-- ... more attributes -->
    </xsd:complexType>
</xsd:element>
```

For complete XSD schema definitions, see:
`src/main/resources/www.liquibase.org/xml/ns/snowflake/liquibase-snowflake-latest.xsd`

Support and Resources

- **Source Code**: GitHub Repository
- **Issue Tracking**: GitHub Issues
- **Test Coverage**: 85% with comprehensive integration tests
- **Documentation**: This guide covers all implemented features

This user guide represents the complete catalog of Snowflake objects and capabilities implemented in the

Liquibase Snowflake Extension as of version 0-SNAPSHOT.